

Семинар по C++ №3

Компилятор

Компилятор

Компилятор - программа, переводящий написанный на одном языке программирования текст, в код на другом языке программирования

Примеры:

g++: C++ -> assembly

JDK: Java -> Java Byte Code

Стадии компиляции

- **Препроцессинг** - текстовая подстановка (-E)
- **Компиляция** - преобразования кода в ассемблер (-S)
- **Ассемблирование** - перевод ассемблерного файла в машинный код - объектный файл (-c)
- **Линковка**
- Сохранить все промежуточные файлы: **--save-temps**

Ассемблерный код

Как посмотреть:

- На сайте godbolt.org
- Скомпилировать исходники:
gcc -S main.cpp
- Дизассемблировать исполняемый файл:
objdump -d -M intel a.out > disasm

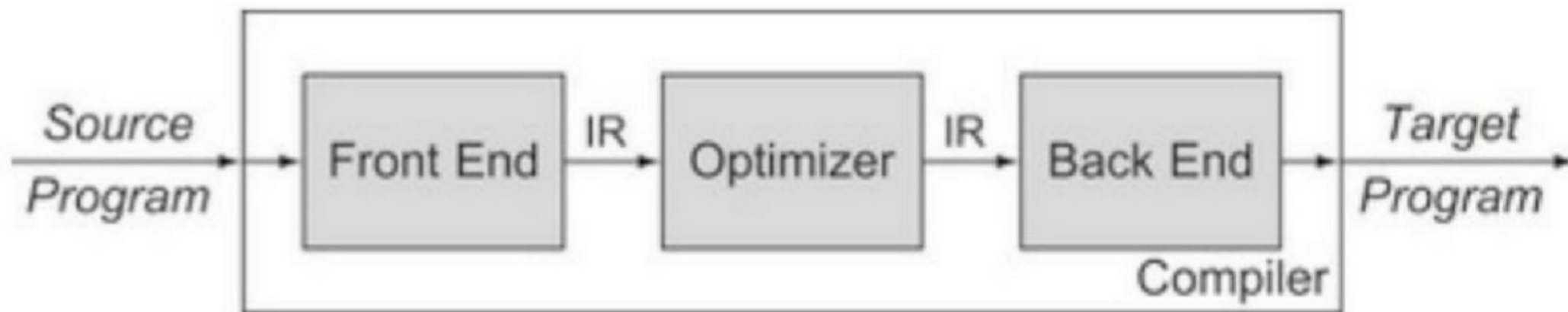
TLDR; Как разбивать программу на файлы?

1. Объявления в `.hpp`
2. Реализация в `.cpp`
3. `#pragma once` or `#ifndef`
4. Линковать в один исполняемый файл:
 `g++ main.cpp my_lib.cpp`
 или
 `g++ main.o my_lib.o`

Интерпретатор

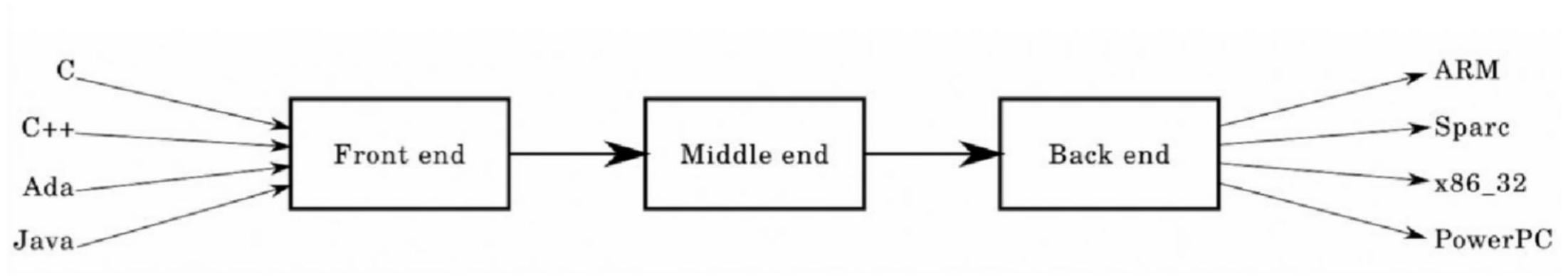
- Интерпретатор - компьютерная программа, которая выполняет инструкции без компиляции в машинный язык
- Примеры:
- Python, Bash, Make

Структура компилятора

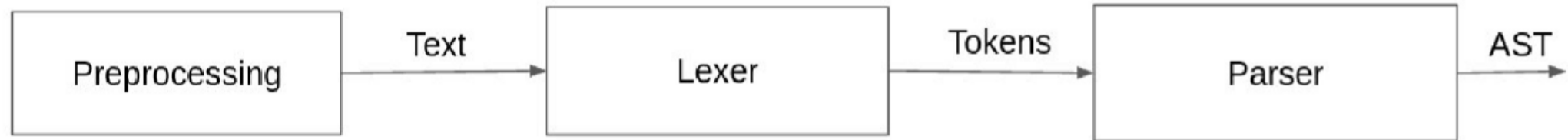


- **Frontend** - трансляция исходного кода в промежуточное представление (**IR** = **I**ntermediate **R**epresentation)
- **Optimizer** - оптимизация промежуточного представления кода
- **Backend** - трансляция промежуточного представления в исполняемый код

Преимущество такой декомпозиции



Frontend



Preprocessing

- `g++ -E main.cpp` - Only preprocess the file
- `g++ -DDEBUG main.cpp` - define something
- It expands:
 - `#include`
 - `#define / #ifdef - #else - #endif / #if`

Lexer

Source code (string)

"a+b*3+4*2"

Lexer

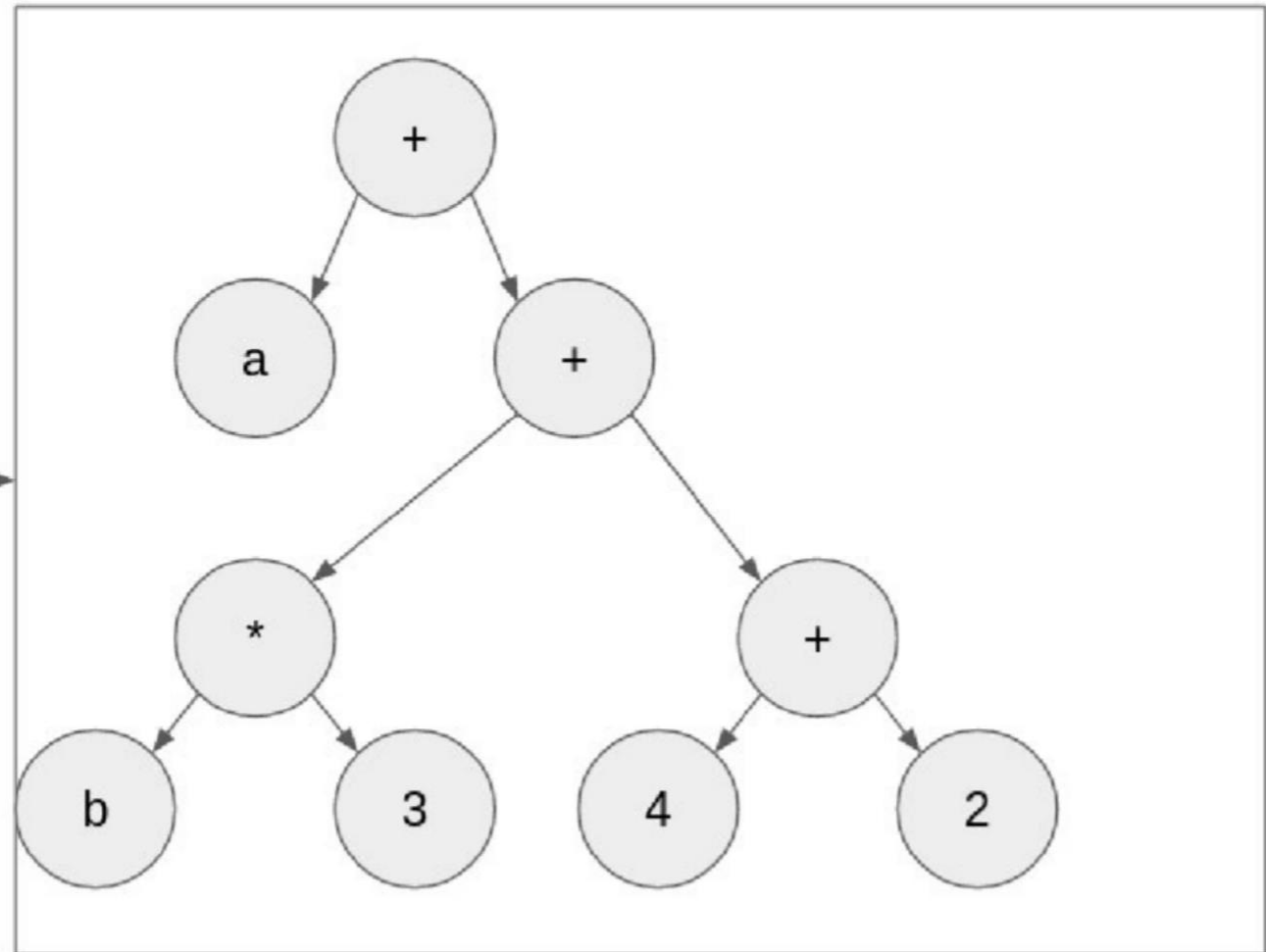
Tokens

[Id "a", Op +, Id b, Op *, Int 3, Op +, Int 4, Op *, Int 2]

Parser

[Id "a", Op +, Id b, Op *, Int 3,
Op +, Int 4, Op *, Int 2]

parser



Python (lexer + parser)

```
def foo(a):  
    a = 2  
    b = 3  
    return a + b
```

lexer + parser

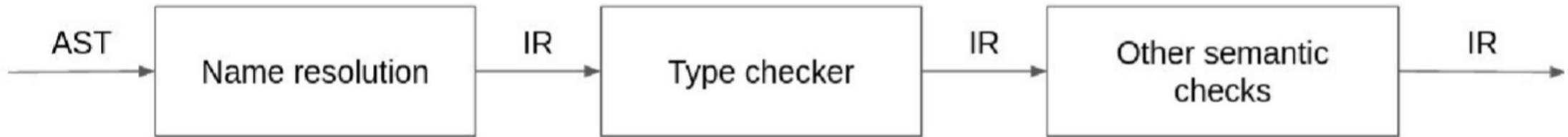


```
FunctionDef(  
    name='foo',  
    args=arguments(  
        posonlyargs=[],  
        args=[  
            arg(arg='a')],  
        kwonlyargs=[],  
        kw_defaults=[],  
        defaults=[]),  
    body=[  
        Assign(  
            targets=[  
                Name(id='a', ctx=Store())],  
            value=Constant(value=2),  
        ),  
        Assign(  
            targets=[  
                Name(id='b', ctx=Store())],  
            value=Constant(value=3),  
        ),  
        Return(  
            value=BinOp(  
                left=Name(id='a', ctx=Load()),  
                op=Add(),  
                right=Name(id='b', ctx=Load()))],  
            decorator_list=[]  
        )  
    ]  
)
```

Middle-end



Semantic analysis

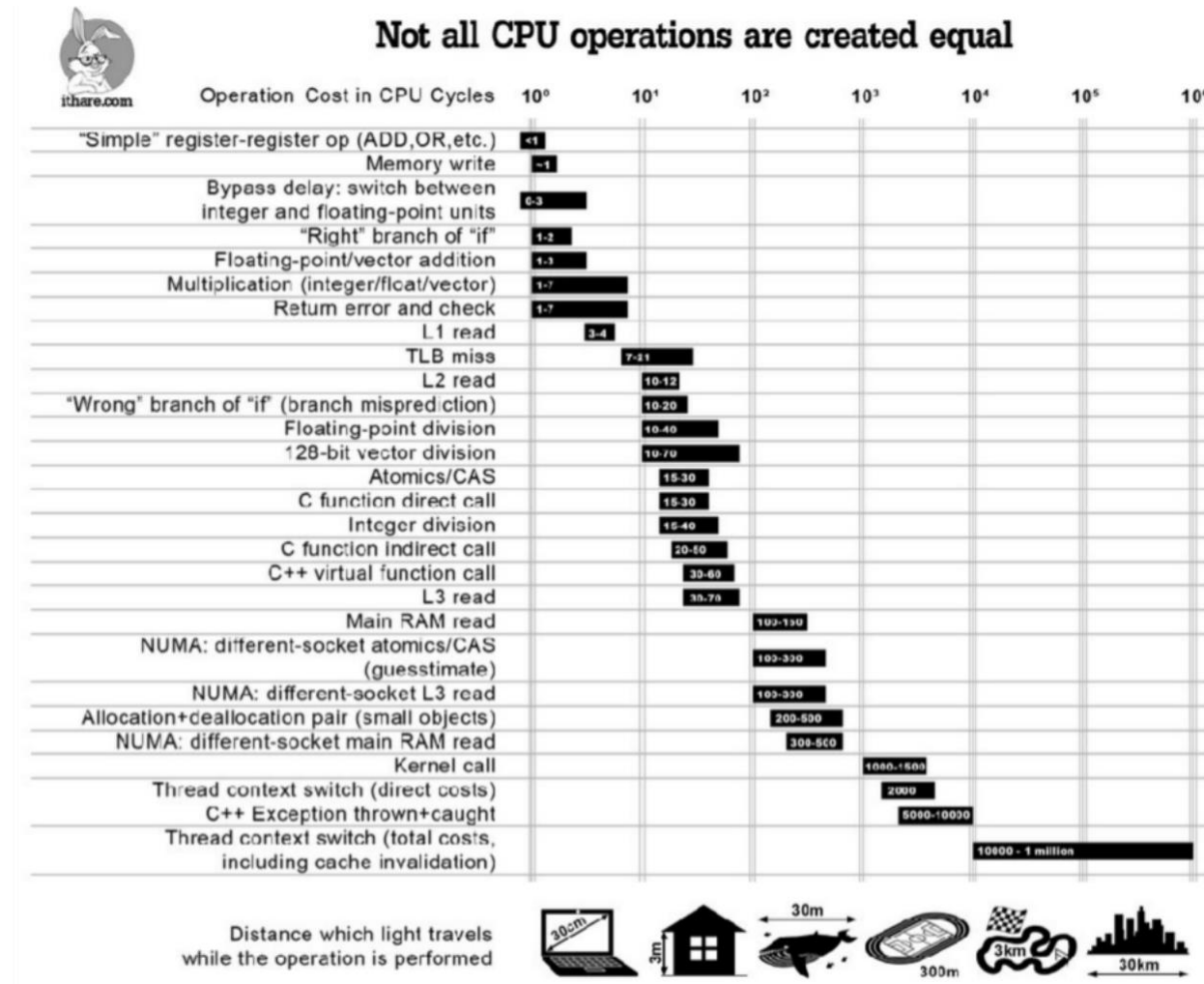


Code optimization

- Peephole optimization
- Loop optimizations
- Inlining
- Constant propagation
- Dead-code elimination

- <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- -O0, -O1, -O2, -O3, -Ofast, -Og
- [The as-if rule](#)

Speed of operations

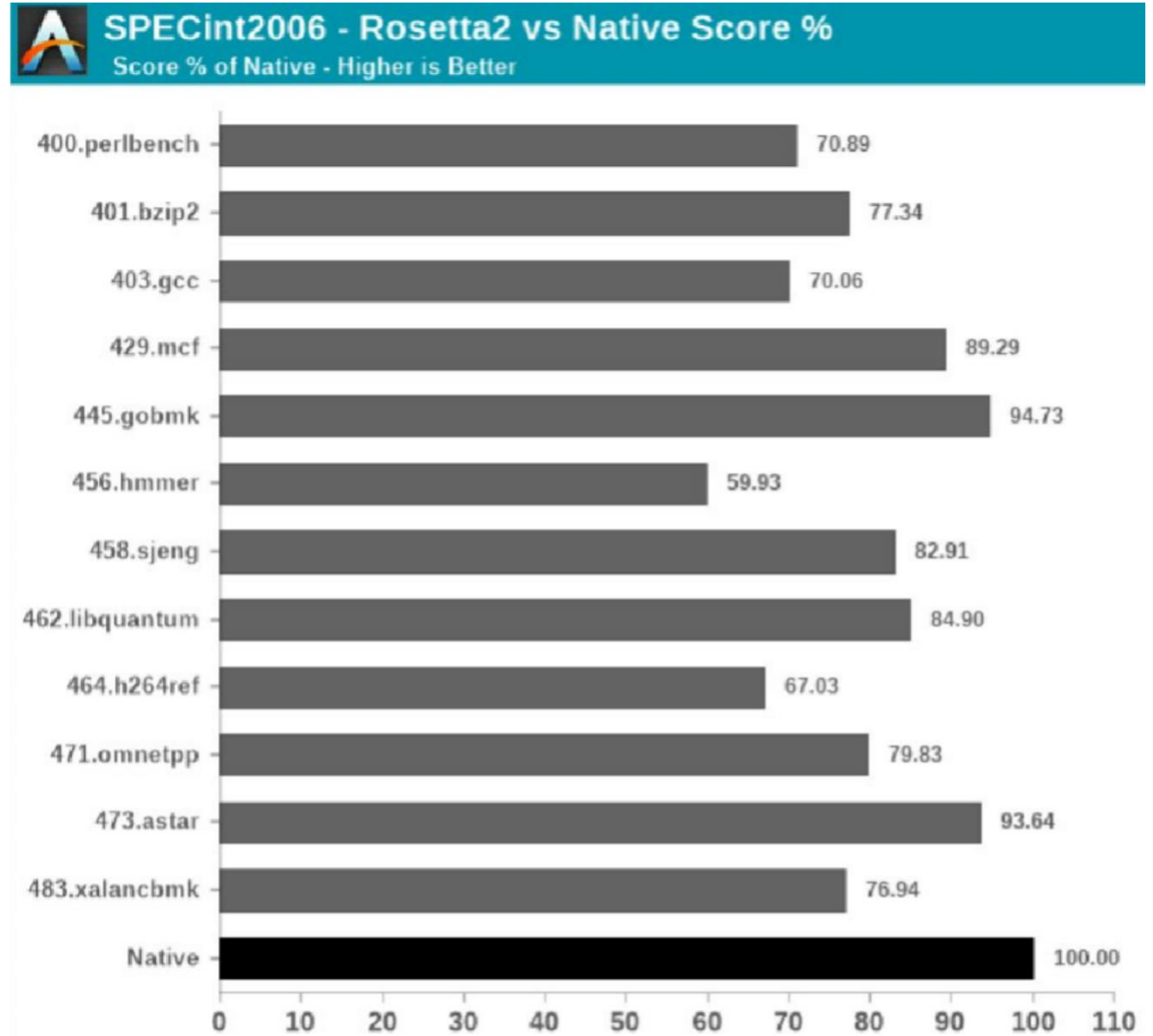


Warnings

- -Wall, -Wextra, -Werror, -Wpedantic
- <https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html>

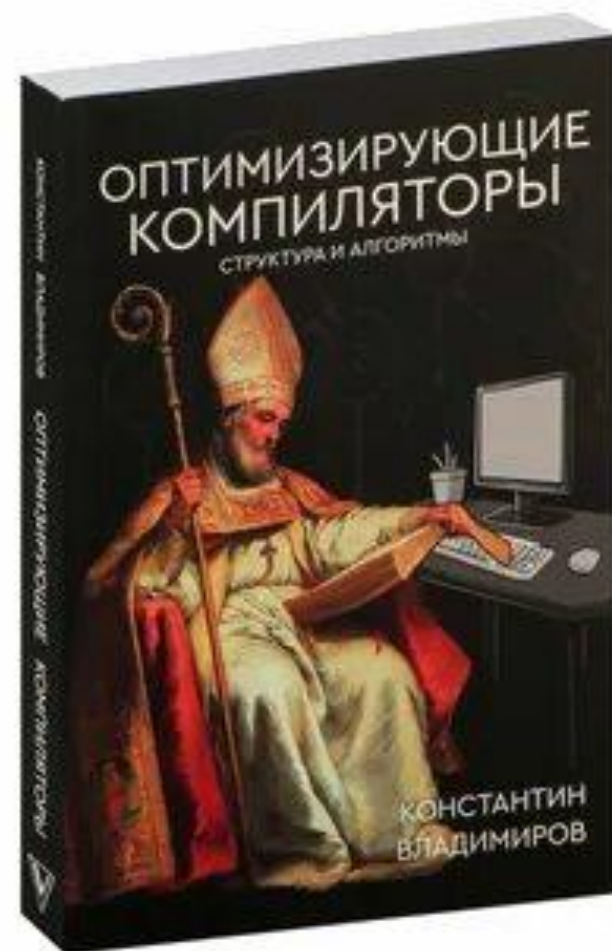
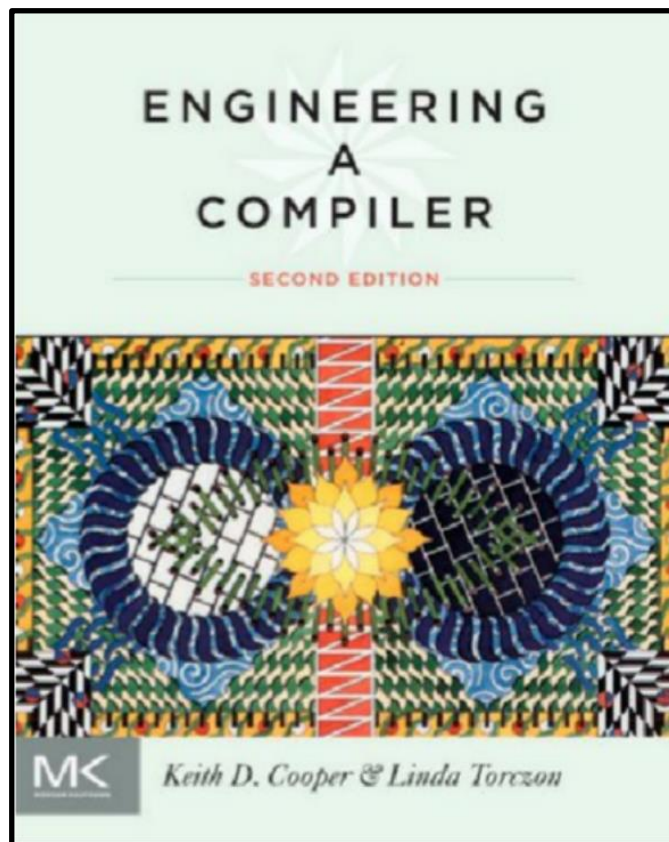
Binary Translation

- Apple Rosetta
- 2006: PowerPC -> Intel
- 2020: Intel -> ARM (M1)



Что почитать

- Dragon Book
- Engineering a compiler
- Оптимизирующие компиляторы



Компиляция

main.cpp

```
void useful_func();
```

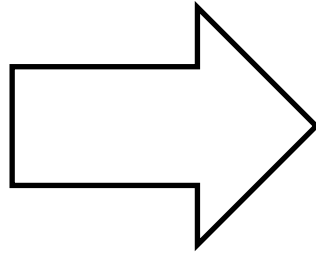
```
void important_func();
```

```
int main() {
```

```
    useful_func();
```

```
    important_func();
```

```
}
```



main.o

0000000000000000 <main>:

0: f3 0f 1e fa endbr64

4: 55 push rbp

5: 48 89 e5 mov rbp, rsp

8: e8 00 00 00 00 call d <main+0xd> # _Z11useful_funcv

d: e8 00 00 00 00 call 12 <main+0x12> > # _Z14important_funcv

12: b8 00 00 00 00 mov eax, 0x0

17: 5d pop rbp

18: c3 ret

g++ -c main.cpp
objdump -M intel -d main.o > dump
или
g++ -S main.cpp

Линковка

main.o

```
0000000000000000 <main>:
 0: f3 0f 1e fa      endbr64
 4: 55               push  rbp
 5: 48 89 e5         mov   rbp,rsp
 8: e8 00 00 00 00   call  d <main+0xd>      # _Z11useful_funcv
d: e8 00 00 00 00   call  12 <main+0x12> > # _Z14important_funcv
12: b8 00 00 00 00   mov   eax,0x0
17: 5d               pop   rbp
18: c3               ret
```

my_lib.o

```
0000000000000000 <_Z11useful_funcv>:
 0: f3 0f 1e fa      endbr64
 4: 55               push  rbp
 5: 48 89 e5         mov   rbp,rsp
.....
21: 90               nop
22: 5d               pop   rbp
23: c3               ret
```

src.o

```
0000000000000000 <_Z14important_funcv>:
 0: f3 0f 1e fa      endbr64
 4: 55               push  rbp
 5: 48 89 e5         mov   rbp,rsp
.....
20: e8 00 00 00 00   call  25 <_Z14important_funcv+0x25> # std::cout
25: c7 45 fc 01 00 00 00 mov   DWORD PTR [rbp-0x4],0x1
2c: e8 00 00 00 00   call  31 <_Z14important_funcv+0x31> # <_Z11useful_funcv>
31: 8b 45 fc         mov   eax,DWORD PTR [rbp-0x4]
34: c9               leave
35: c3               ret
```

g++ main.o my_lib.o src.o

Линковка

a.out

0000000000001149 <main>:

```
1149: f3 0f 1e fa      endbr64
114d: 55              push  rbp
114e: 48 89 e5        mov   rbp, rsp
1151: e8 42 00 00 00  call 1198 <_Z11useful_funcv>
1156: e8 07 00 00 00  call1162 <_Z14important_funcv>
115b: b8 00 00 00 00  mov   eax, 0x0
1160: 5d              pop   rbp
1161: c3              ret
```

0000000000001198 <_Z11useful_funcv>:

```
1198: f3 0f 1e fa      endbr64
119c: 55              push  rbp
119d: 48 89 e5        mov   rbp, rsp
11a0: 48 8d 05 70 0e 00 00 lea   rax, [rip+0xe70]
11a7: 48 89 c6        mov   rsi, rax
11aa: 48 8d 05 8f 2e 00 00 lea   rax, [rip+0x2e8f] # 4040 <_ZSt4cout....>
11b1: 48 89 c7        mov   rdi, rax
11b4: e8 97 fe ff ff  call 1050 <_ZStlsISt11char_traitsIcE....>
11b9: 90              nop
11ba: 5d              pop   rbp
11bb: c3              ret
```

0000000000001162 <_Z14important_funcv>:

```
1162: f3 0f 1e fa      endbr64
1166: 55              push  rbp
1167: 48 89 e5        mov   rbp, rsp
116a: 48 83 ec 10     sub   rsp, 0x10
116e: 48 8d 05 8f 0e 00 00 lea   rax, [rip+0xe8f] # 2004 <_IO_stdin_used+0x4>
1175: 48 89 c6        mov   rsi, rax
1178: 48 8d 05 c1 2e 00 00 lea   rax, [rip+0x2ec1] # 4040 _ZSt4cout....
117f: 48 89 c7        mov   rdi, rax
1182: e8 c9 fe ff ff  call 1050 <_ZStlsISt11char_traitsIcE....>
1187: c7 45 fc 01 00 00 00 mov   DWORD PTR [rbp-0x4], 0x1
118e: e8 05 00 00 00  call1198 <_Z11useful_funcv>
1193: 8b 45 fc        mov   eax, DWORD PTR [rbp-0x4]
1196: c9              leave
1197: c3              ret
```

g++ main.o my_lib.o src.o ; objdump -M intel -d a.out > dump

Таблица символов

main.o: file format elf64-x86-64

SYMBOL TABLE:

```
0000000000000000 | df *ABS* 0000000000000000 main.cpp
0000000000000000 | d .text 0000000000000000 .text
0000000000000000 g F .text 0000000000000019 main
0000000000000000 *UND* 0000000000000000 _Z11useful_funcv
0000000000000000 *UND* 0000000000000000 _Z14important_funcv
```

objdump -t main.o

src.o: file format elf64-x86-64

SYMBOL TABLE:

```
0000000000000000 | df *ABS* 0000000000000000 src.cpp
0000000000000000 | d .text 0000000000000000 .text
.....
0000000000000000 g F .text 0000000000000036 _Z14important_funcv
0000000000000000 *UND* 0000000000000000 _ZSt4cout
0000000000000000 *UND* 0000000000000000 _ZStlsSt11char_traitslcE...
0000000000000000 *UND* 0000000000000000 _Z11useful_funcv
```

objdump -t src.o

Таблица символов

a.out: file format elf64-x86-64

SYMBOL TABLE:

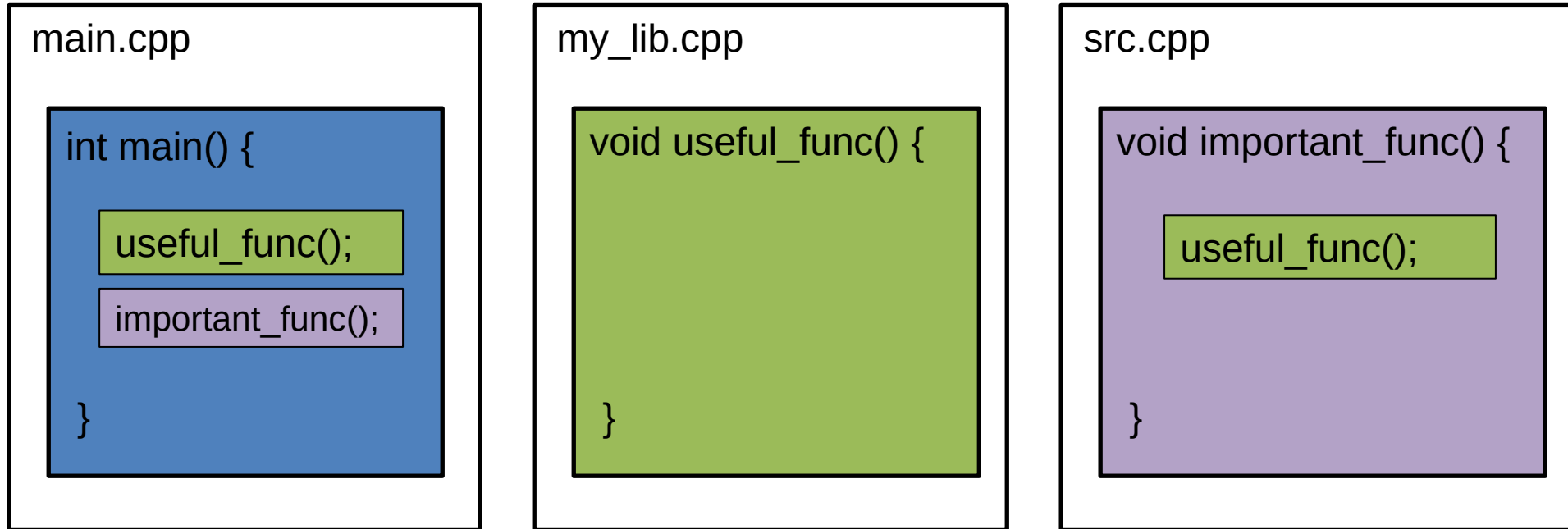
.....

0000000000001149 g	F .text	0000000000000019
0000000000001198 g	F .text	0000000000000024
0000000000001162 g	F .text	0000000000000036
0000000000004040 g	O .bss	0000000000000110

main
<u>_Z11useful_funcv</u>
<u>_Z14important_funcv</u>
<u>_ZSt4cout...</u>

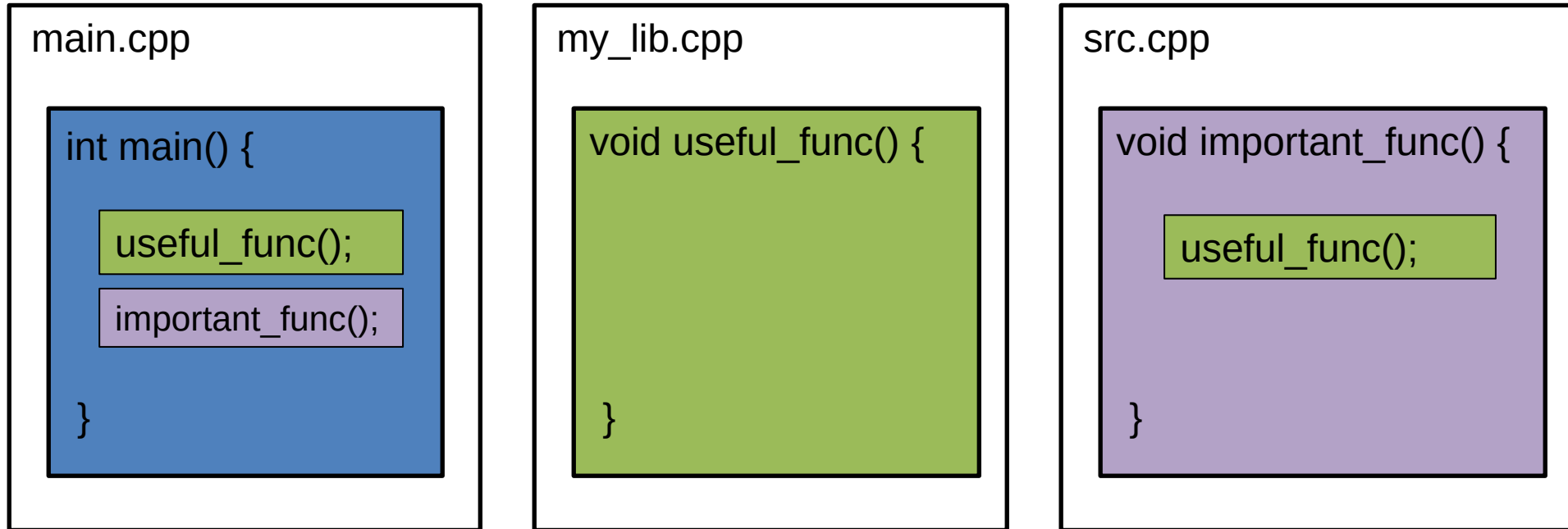
objdump -t a.out

Линковка (версия 1)



```
g++ -c main.cpp  
g++ -c my_lib.cpp  
g++ -c src.cpp  
g++ main.o my_lib.o src.o
```

Линковка (версия 1)



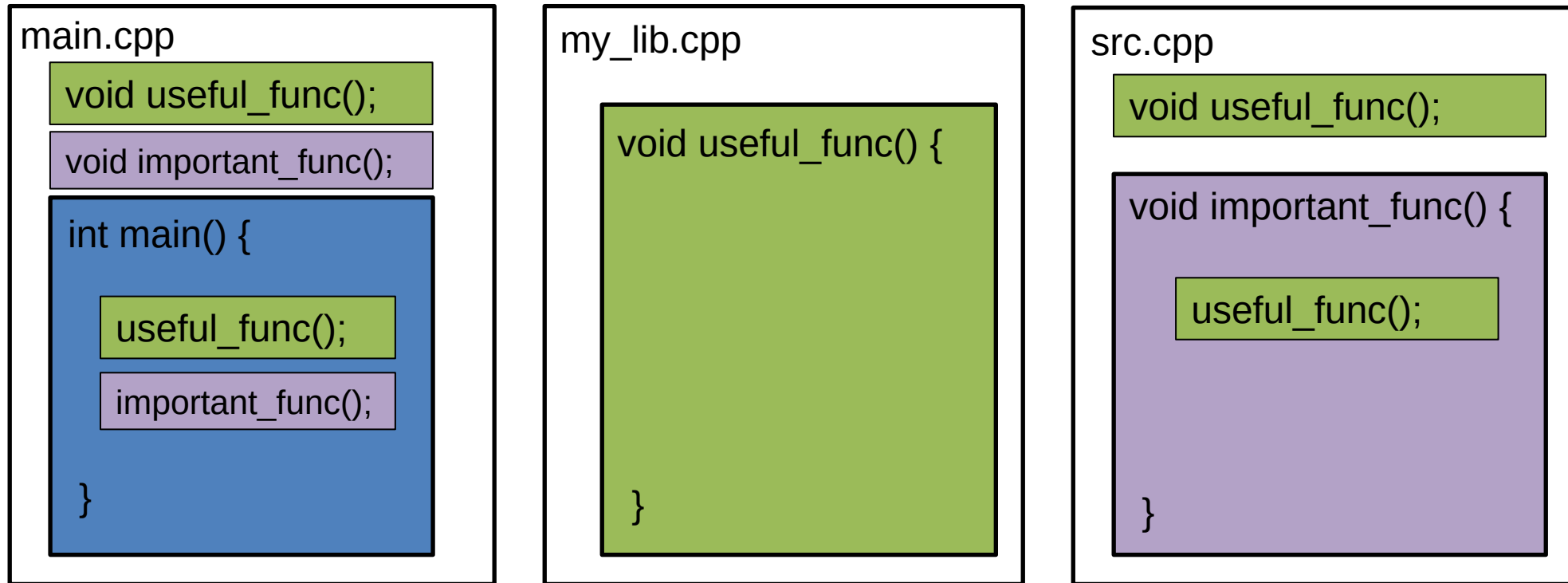
g++ -c main.cpp – **CE: 'useful_func' was not declared in this scope**

g++ -c my_lib.cpp - **OK**

g++ -c src.cpp – **CE: 'useful_func' was not declared in this scope**

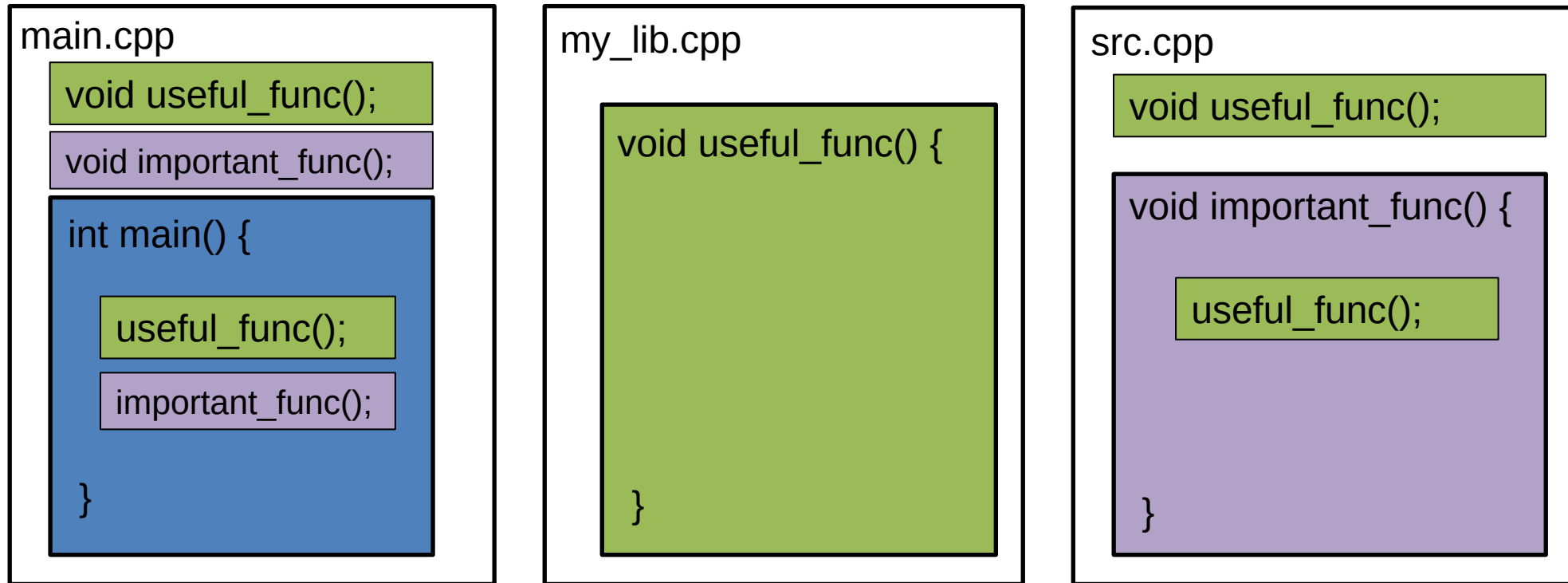
g++ main.o my_lib.o src.o – **X**

Линковка (версия 2)



```
g++ -c main.cpp
g++ -c my_lib.cpp
g++ -c src.cpp
g++ main.o my_lib.o src.o
```

Линковка (версия 2)



g++ -c main.cpp – **OK**

g++ -c my_lib.cpp – **OK**

g++ -c src.cpp – **OK**

g++ main.o my_lib.o src.o – **OK**

Линковка (версия 3)

main.cpp

```
#include "my_lib.hpp"
```

```
#include "src.hpp"
```

```
int main() {
```

```
    useful_func();
```

```
    important_func();
```

```
}
```

my_lib.cpp

```
void useful_func() {
```

```
}
```

src.cpp

```
#include "my_lib.hpp"
```

```
void important_func() {
```

```
    useful_func();
```

```
}
```

src.hpp

```
void important_func();
```

my_lib.hpp

```
void useful_func();
```

```
g++ -c main.cpp
g++ -c my_lib.cpp
g++ -c src.cpp
g++ main.o my_lib.o src.o
```

Линковка (версия 3)

main.cpp

```
#include "my_lib.hpp"
```

```
#include "src.hpp"
```

```
int main() {
```

```
    useful_func();
```

```
    important_func();
```

```
}
```

my_lib.cpp

```
void useful_func() {
```

```
}
```

src.cpp

```
#include "my_lib.hpp"
```

```
void important_func() {
```

```
    useful_func();
```

```
}
```

src.hpp

```
void important_func();
```

my_lib.hpp

```
void useful_func();
```

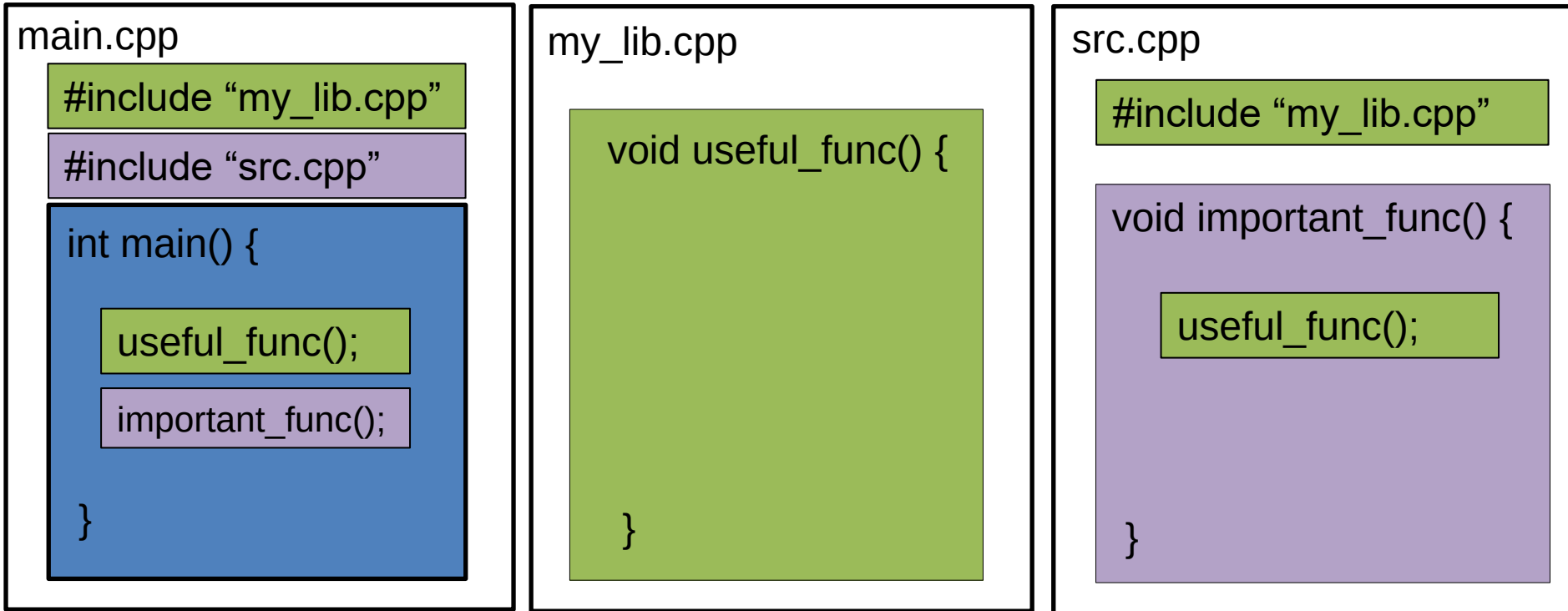
g++ -c main.cpp – OK

g++ -c my_lib.cpp – OK

g++ -c src.cpp – OK

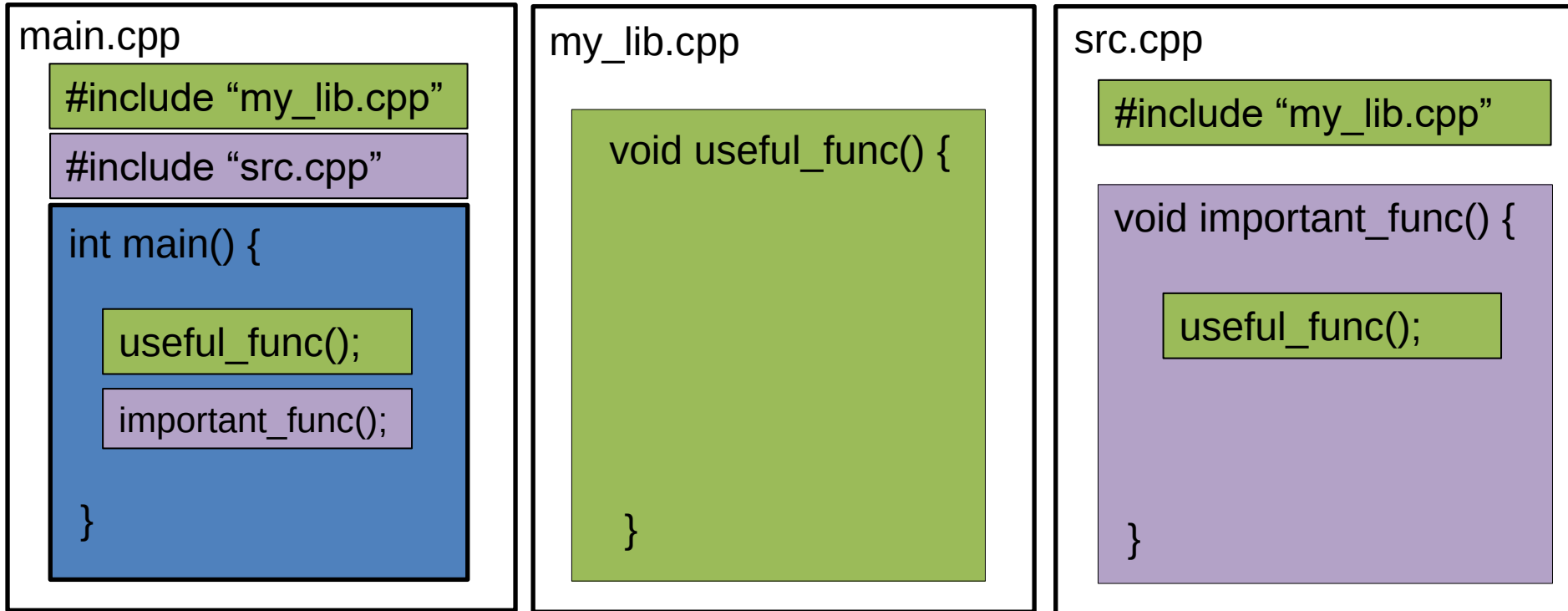
g++ main.o my_lib.o src.o – OK

Линковка (версия 4)



```
g++ -c main.cpp
g++ -c my_lib.cpp
g++ -c src.cpp
g++ main.o my_lib.o src.o
```

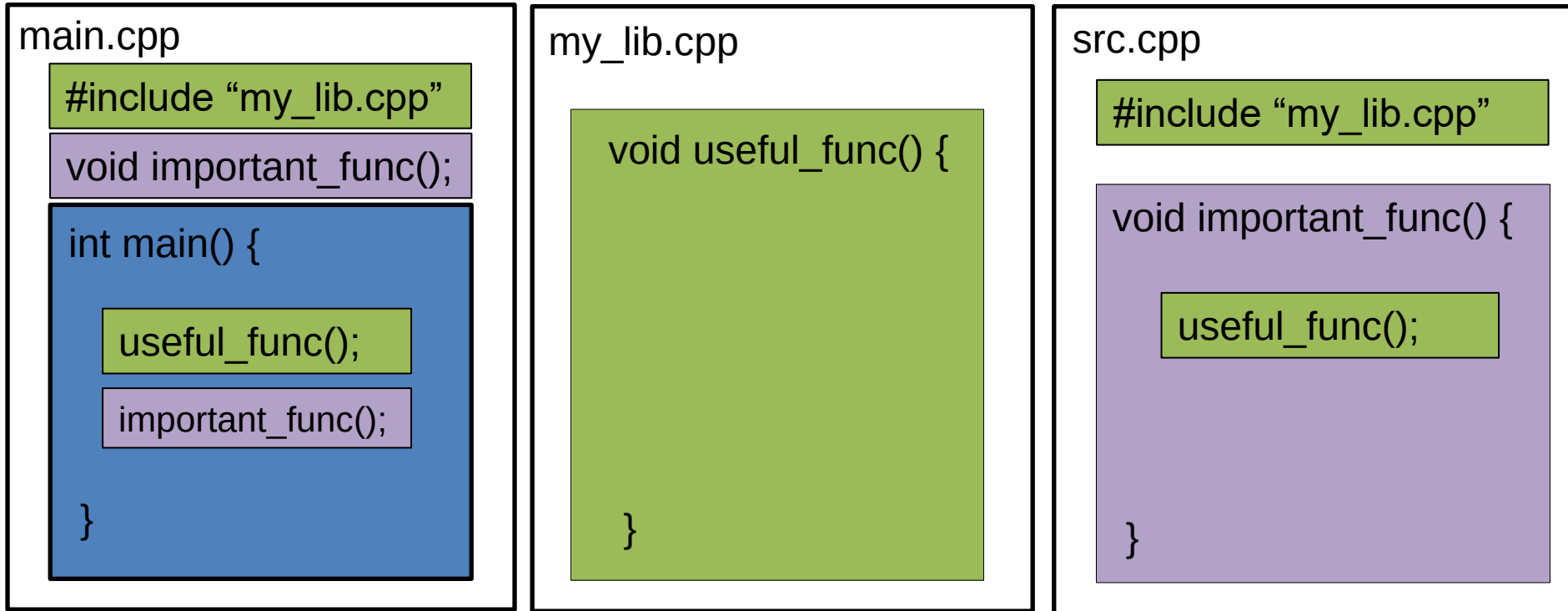

Линковка (версия 4)



```
g++ -c main.cpp - X
g++ -c my_lib.cpp - OK
g++ -c src.cpp - OK
g++ main.o my_lib.o src.o - X
```

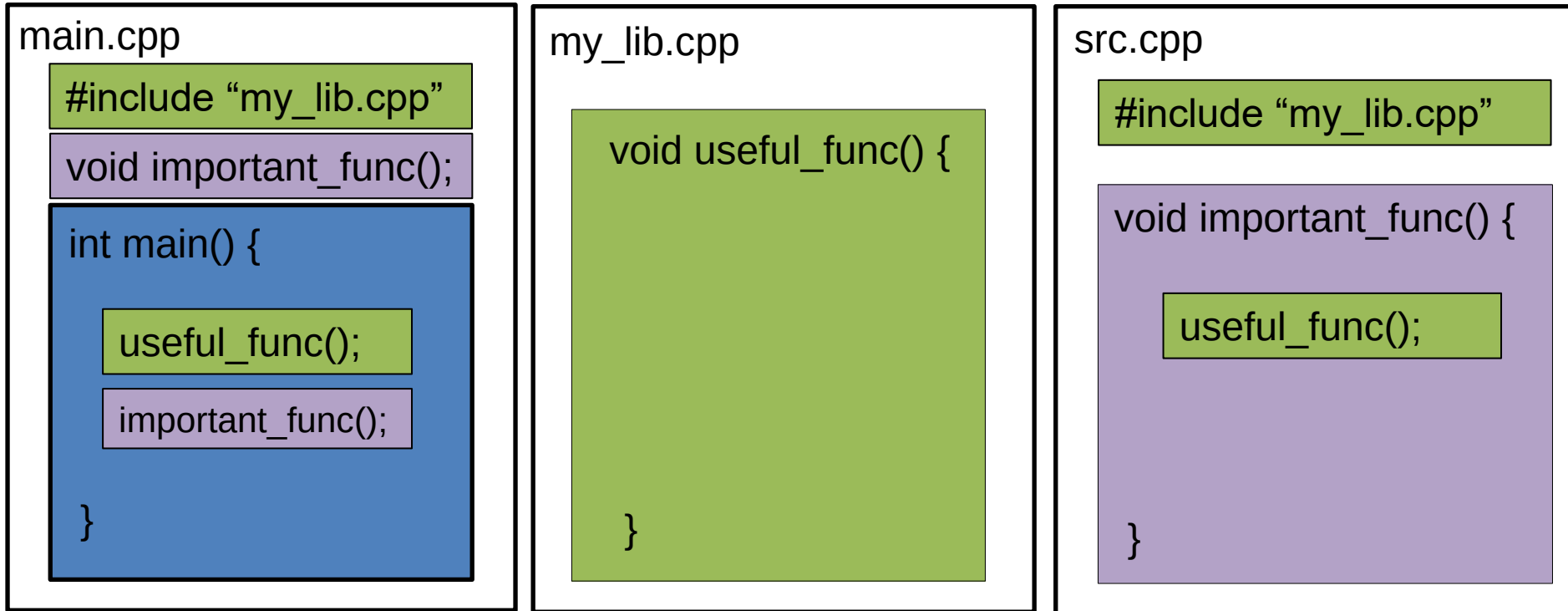
```
In file included from src.cpp:2,
                 from main.cpp:2:
my_lib.cpp:4:6: error: redefinition of 'void useful_func()'
4 | void useful_func() {
  | ^~~~~~
In file included from main.cpp:1:
my_lib.cpp:4:6: note: 'void useful_func()' previously defined here
4 | void useful_func() {
```

Линковка (версия 5)



```
g++ -c main.cpp
g++ -c my_lib.cpp
g++ -c src.cpp
g++ main.o my_lib.o src.o
```

Линковка (версия 5)



```
g++ -c main.cpp – OK
g++ -c my_lib.cpp – OK
g++ -c src.cpp – OK
g++ main.o my_lib.o src.o – X
```

```
/usr/bin/ld: src.o: in function `useful_func()':
src.cpp:(.text+0x0): multiple definition of `useful_func()';
main.o:main.cpp:(.text+0x0): first defined here
collect2: error: ld returned 1 exit status
```